

HERRAMIENTAS DE PROGRAMACIÓN WEB DEL SERVIDOR

Tema 4 — Seguridad y Buenas Prácticas en Programación Web

Seguridad en Aplicaciones Web

Fundamentos, amenazas y arquitectura segura

1. Introducción

La seguridad en aplicaciones web es el conjunto de medidas, técnicas y buenas prácticas orientadas a proteger los sitios y sistemas que se ejecutan en un servidor frente a accesos no autorizados, robo de información, interrupción del servicio o manipulación de datos. A medida que las aplicaciones web gestionan información cada vez más sensible (datos personales, credenciales, transacciones financieras), la seguridad deja de ser un aspecto opcional y se convierte en un requisito indispensable del ciclo de vida del desarrollo de software.

En un entorno cliente-servidor, la superficie de ataque incluye el propio código de la aplicación, el servidor donde se aloja, la base de datos, las comunicaciones de red y, en muchos casos, el propio navegador del usuario. Por ello, la seguridad debe abordarse de forma integral, no como una capa añadida al final del desarrollo, sino como parte del diseño desde el inicio (principio de "seguridad por diseño" o security by design).

2. ¿Por qué es importante la seguridad web?

- **Protección de datos:** evita el robo o filtración de información personal, financiera o corporativa.
- **Continuidad del negocio:** un ataque exitoso puede dejar fuera de servicio una aplicación durante horas o días, generando pérdidas económicas.
- **Reputación:** una brecha de seguridad daña la confianza de los usuarios y clientes de forma difícil de recuperar.
- **Cumplimiento legal:** normativas como el RGPD exigen medidas mínimas de seguridad en el tratamiento de datos personales, con sanciones económicas en caso de incumplimiento (Parlamento Europeo y Consejo de la Unión Europea, 2016).
- **Coste del error:** corregir una vulnerabilidad detectada en producción es mucho más costoso que prevenirla durante el diseño o la codificación.

3. Principios fundamentales de la seguridad de la información

La seguridad de cualquier sistema, incluidas las aplicaciones web, se apoya tradicionalmente en tres pilares conocidos como la tríada CIA, presentes en guías de referencia como los principios de seguridad de Mozilla (Mozilla, s. f.):

- **Confidencialidad:** garantizar que la información solo sea accesible para quienes están autorizados a verla (por ejemplo, mediante cifrado y control de acceso).
- **Integridad:** asegurar que los datos no sean alterados de forma no autorizada, ni accidental ni maliciosamente.
- **Disponibilidad:** garantizar que el sistema y la información estén accesibles cuando los usuarios legítimos los necesiten.

A estos tres principios se suelen añadir otros dos, especialmente relevantes en aplicaciones web:

- **Autenticación:** verificar que un usuario es quien dice ser (contraseñas, tokens, autenticación multifactor).
- **Autorización:** una vez autenticado, controlar qué acciones y recursos puede utilizar cada usuario según su rol o permisos.

4. Principales amenazas y vectores de ataque

Las amenazas a las que se enfrenta una aplicación web pueden clasificarse según el punto del sistema que explotan, como recoge de forma sistemática el OWASP Top 10 (OWASP Foundation, 2021):

4.1 Amenazas a nivel de aplicación

- **Inyección de código:** SQL, comandos del sistema operativo o código script insertado a través de entradas de usuario no validadas.
- **Cross-Site Scripting (XSS):** inserción de scripts maliciosos que se ejecutan en el navegador de otros usuarios.
- **Cross-Site Request Forgery (CSRF):** engañar al navegador de un usuario autenticado para que realice acciones no deseadas.
- **Gestión de sesiones débil:** identificadores de sesión predecibles, ausencia de expiración o falta de cifrado en las cookies.

4.2 Amenazas a nivel de infraestructura

- **Configuración insegura del servidor:** puertos abiertos innecesarios, servicios desactualizados, credenciales por defecto.
- **Ataques de denegación de servicio (DoS/DDoS):** saturación del servidor mediante peticiones masivas.
- **Falta de cifrado en tránsito:** uso de HTTP en lugar de HTTPS, exponiendo datos a interceptación (ataques "man in the middle").

4.3 Amenazas relacionadas con el factor humano

- **Ingeniería social y phishing:** engaño a usuarios o administradores para obtener credenciales.
- **Contraseñas débiles o reutilizadas:** facilitan el acceso no autorizado mediante fuerza bruta o diccionarios.

5. Hacia una arquitectura segura

Diseñar una aplicación web segura implica aplicar de forma combinada varios principios arquitectónicos, entre los que destacan los siguientes (Mozilla, s. f.; Stuttard & Pinto, 2011):

- **Defensa en profundidad:** aplicar múltiples capas de seguridad (red, servidor, aplicación, base de datos) de modo que si una falla, las demás sigan protegiendo el sistema.

- **Principio de mínimo privilegio:** cada componente, proceso o usuario debe disponer únicamente de los permisos estrictamente necesarios para su función.
- **Superficie de ataque mínima:** deshabilitar servicios, puertos, funcionalidades o endpoints que no se utilicen.
- **Fail secure:** ante un fallo o excepción, el sistema debe quedar en un estado seguro (por ejemplo, denegando el acceso) en lugar de un estado permisivo.
- **Separación de entornos:** mantener entornos de desarrollo, pruebas y producción claramente diferenciados y con distintas credenciales.

En la segunda parte de este trabajo se abordará en detalle el OWASP Top 10, el listado de referencia internacional sobre los riesgos de seguridad más críticos en aplicaciones web, junto con sus contramedidas, así como un conjunto de buenas prácticas de codificación segura aplicables al desarrollo diario.

Referencias bibliográficas

- Mozilla. (s. f.). Web Security Guidelines. Mozilla Infosec. https://infosec.mozilla.org/guidelines/web_security
- National Institute of Standards and Technology. (2020). Digital Identity Guidelines: Authentication and Lifecycle Management (NIST SP 800-63B). U.S. Department of Commerce. <https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final>
- OWASP Foundation. (2021). OWASP Top 10:2021 — The ten most critical web application security risks. <https://owasp.org/Top10/2021/>
- OWASP Foundation. (s. f.). OWASP Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>
- Parlamento europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD). Diario Oficial de la Unión Europea, L 119. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=celex%3A32016R0679>
- Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws (2.ª ed.). Wiley.

HERRAMIENTAS DE PROGRAMACIÓN WEB DEL SERVIDOR

Tema 4 — Seguridad y Buenas Prácticas en Programación Web

OWASP Top 10 y Buenas Prácticas

Riesgos críticos, contramedidas y codificación segura

1. ¿Qué es OWASP?

OWASP (Open Worldwide Application Security Project) es una fundación sin ánimo de lucro dedicada a mejorar la seguridad del software. Entre sus proyectos más conocidos se encuentra el OWASP Top 10, un documento de referencia que resume los diez riesgos de seguridad más críticos en aplicaciones web, elaborado a partir del análisis de datos reales de vulnerabilidades reportadas por organizaciones de todo el mundo (OWASP Foundation, 2021). Se actualiza periódicamente para reflejar la evolución de las amenazas y sirve como guía tanto para desarrolladores como para auditores de seguridad.

2. OWASP Top 10 y sus contramedidas

La siguiente tabla resume, de forma orientativa, las diez categorías de riesgo y las principales medidas para mitigarlas:

Riesgo	Descripción	Contramedida principal
A01	Pérdida de control de acceso (Broken Access Control)	Aplicar el principio de mínimo privilegio, verificar permisos en el servidor en cada petición y denegar por defecto.
A02	Fallos criptográficos (Cryptographic Failures)	Cifrar datos sensibles en tránsito (TLS) y en reposo, usar algoritmos actualizados y no almacenar contraseñas en texto plano.
A03	Inyección (Injection)	Usar consultas parametrizadas o ORM, validar y sanear toda entrada de usuario, evitar concatenar código dinámico.
A04	Diseño inseguro (Insecure Design)	Incorporar modelado de amenazas y requisitos de seguridad desde las fases iniciales del diseño.
A05	Configuración de seguridad incorrecta	Eliminar cuentas y servicios por defecto, aplicar bastionado (hardening) y mantener el software actualizado.
A06	Componentes vulnerables y desactualizados	Inventariar dependencias, aplicar parches y usar herramientas de análisis de vulnerabilidades (SCA).
A07	Fallos de identificación y autenticación	Implementar autenticación multifactor, políticas de contraseñas robustas y control de intentos fallidos.
A08	Fallos de integridad de software y datos	Verificar firmas digitales, usar pipelines de CI/CD seguros y controlar el origen de las dependencias.
A09	Fallos de registro y monitorización	Registrar eventos de seguridad relevantes, centralizar logs y establecer alertas ante actividad sospechosa.
A10	Server-Side Request Forgery (SSRF)	Validar y limitar las URLs a las que el servidor puede acceder, usando listas blancas de destinos permitidos.

Aunque el orden y la redacción exacta de cada categoría pueden variar entre versiones del documento, el objetivo se mantiene: ofrecer una lista priorizada que ayude a los equipos de desarrollo a enfocar sus esfuerzos de seguridad en los riesgos con mayor impacto y frecuencia real (OWASP Foundation, 2021).

3. Buenas prácticas de codificación web

Más allá de conocer los riesgos del OWASP Top 10, es fundamental incorporar hábitos de codificación segura en el día a día del desarrollo. A continuación se presentan las prácticas más relevantes:

3.1 Validación y saneamiento de entradas

- **Validar en el servidor:** nunca confiar únicamente en la validación realizada en el cliente (JavaScript), ya que puede eludirse fácilmente.
- **Listas blancas:** preferir la validación por listas blancas (qué se permite) frente a listas negras (qué se bloquea).
- **Escapado de salida:** codificar correctamente los datos antes de mostrarlos en HTML, JavaScript o consultas SQL para evitar inyecciones.

3.2 Gestión segura de la autenticación y sesiones

- **Contraseñas:** almacenar siempre con funciones de hash específicas para contraseñas (bcrypt, Argon2), nunca en texto plano ni con hash simple, siguiendo recomendaciones como las de NIST SP 800-63B (National Institute of Standards and Technology, 2020).
- **Cookies de sesión:** usar los atributos HttpOnly, Secure y SameSite para reducir el riesgo de robo de sesión (OWASP Foundation, s. f.).
- **Expiración y renovación:** establecer tiempos de expiración razonables y regenerar el identificador de sesión tras el inicio de sesión.

3.3 Cifrado y protección de datos

- **HTTPS obligatorio:** forzar el uso de TLS en toda la aplicación, redirigiendo automáticamente el tráfico HTTP (Mozilla, s. f.).
- **Cifrado de datos sensibles:** proteger en la base de datos la información especialmente sensible (datos personales, tarjetas, tokens), en línea con lo exigido por el RGPD (Parlamento Europeo y Consejo de la Unión Europea, 2016).
- **Gestión de secretos:** no incluir contraseñas, claves de API o tokens directamente en el código fuente ni en el repositorio.

3.4 Control de acceso y principio de mínimo privilegio

- **Verificación en cada petición:** comprobar permisos en el servidor para cada acción, no solo al iniciar sesión.
- **Denegar por defecto:** cualquier recurso debe ser inaccesible salvo que exista una regla explícita que lo permita.

3.5 Manejo de errores y registro de eventos

- **Mensajes de error genéricos:** evitar mostrar al usuario detalles técnicos (rutas, versiones, trazas de pila) que puedan ayudar a un atacante.
- **Registro (logging):** registrar eventos de seguridad relevantes (accesos fallidos, cambios de permisos) sin almacenar datos sensibles en los logs.

3.6 Actualización y gestión de dependencias

- **Mantener el software actualizado:** aplicar parches de seguridad del framework, del servidor y de las librerías utilizadas.
- **Auditoría de dependencias:** revisar periódicamente las librerías de terceros en busca de vulnerabilidades conocidas.

4. Conclusión

La seguridad en aplicaciones web no es un producto final, sino un proceso continuo que debe integrarse en todas las fases del desarrollo: diseño, codificación, pruebas y mantenimiento. Conocer los riesgos descritos en el OWASP Top 10 y aplicar de forma sistemática las buenas prácticas de codificación segura permite reducir drásticamente la probabilidad de sufrir un incidente de seguridad, protegiendo tanto a la organización como a los usuarios finales de la aplicación.

Referencias bibliográficas

- Mozilla. (s. f.). Web Security Guidelines. Mozilla Infosec. https://infosec.mozilla.org/guidelines/web_security
- National Institute of Standards and Technology. (2020). Digital Identity Guidelines: Authentication and Lifecycle Management (NIST SP 800-63B). U.S. Department of Commerce. <https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final>
- OWASP Foundation. (2021). OWASP Top 10:2021 — The ten most critical web application security risks. <https://owasp.org/Top10/2021/>
- OWASP Foundation. (s. f.). OWASP Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>
- Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD). Diario Oficial de la Unión Europea, L 119. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=celex%3A32016R0679>
- Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws (2.ª ed.). Wiley.

Acceso a Datos y Patrones Arquitecturales

Tema 2

Objetos Integrados y Gestión de Estado en Aplicaciones Web

1. El objeto Request

El objeto Request representa la solicitud que un cliente (navegador) envía al servidor web. Es el punto de entrada de toda la información que el usuario transmite en una petición HTTP.

Características principales

- Contiene los datos enviados mediante formularios (método GET o POST).
- Incluye las cabeceras HTTP (headers): tipo de navegador, idioma, cookies enviadas, tipo de contenido, etc.
- Permite acceder a parámetros de la URL (query string).
- Proporciona información sobre la ruta solicitada, el método HTTP usado y la dirección IP del cliente.

Usos típicos

- Leer datos introducidos en un formulario (Request.Form o Request.QueryString en ASP clásico/ASP.NET; req.body o req.query en Node/Express).
- Recuperar cookies enviadas por el cliente.
- Detectar el tipo de dispositivo o navegador para adaptar la respuesta.

2. El objeto Response

El objeto Response representa la respuesta que el servidor envía de vuelta al cliente. Es el objeto encargado de construir lo que finalmente ve el usuario en su navegador.

Características principales

- Permite escribir contenido HTML, texto o datos (JSON, XML) hacia el cliente.
- Controla las cabeceras de respuesta (tipo de contenido, códigos de estado HTTP, cookies a enviar).
- Permite redirecciones (Response.Redirect) hacia otra página o recurso.
- Puede finalizar la ejecución de la página (Response.End) o limpiar el buffer de salida (Response.Clear).

Usos típicos

- Enviar el resultado de una consulta al usuario.

- Establecer cookies en el navegador del cliente.
- Redirigir al usuario tras un login exitoso.
- Configurar códigos de estado (200 OK, 404 Not Found, 500 Error, etc.).

3. El objeto Session

El objeto Session permite mantener información específica de un usuario a lo largo de múltiples peticiones, ya que HTTP es un protocolo sin estado (stateless).

Características principales

- Cada usuario que se conecta a la aplicación tiene su propia sesión, identificada normalmente mediante una cookie con un ID único (Session ID).
- Los datos almacenados en la sesión persisten mientras el usuario navega por la aplicación, hasta que expira (por inactividad) o se cierra explícitamente.
- Es ideal para guardar información como usuario autenticado, carrito de compra o preferencias temporales.

Consideraciones

- Consume recursos del servidor (memoria), por lo que en aplicaciones con muchos usuarios concurrentes debe gestionarse con cuidado (sesiones distribuidas, almacenamiento en base de datos o en memoria compartida tipo Redis).
- Tiene un tiempo de expiración configurable (timeout).

4. Los objetos Application y Page

Objeto Application

- Representa el estado global de toda la aplicación web, compartido por todos los usuarios que acceden a ella.
- Se utiliza para almacenar datos comunes a todas las sesiones: contadores de visitas, configuración global, caché de datos compartidos.
- Al ser compartido, requiere sincronización (bloqueo/lock) cuando varios usuarios intentan modificarlo simultáneamente, para evitar condiciones de carrera.

Objeto Page

- Representa la página individual que se está procesando en el servidor en el modelo de programación por eventos (por ejemplo, en ASP.NET Web Forms).
- Gestiona el ciclo de vida de la página: inicialización, carga, renderizado y descarga.
- Permite acceder a los controles de la página y gestionar su estado durante el procesamiento de la solicitud.

Resumen comparativo del alcance (scope)

Objeto	Alcance	Duración
Request	Una sola petición	Milisegundos (dura la petición)
Response	Una sola petición	Milisegundos (dura la petición)
Session	Un usuario específico	Mientras dure su sesión activa
Application	Todos los usuarios	Toda la vida de la aplicación
Page	Una página en un momento dado	Ciclo de vida de esa página

Tema 3

Patrones de Arquitectura para Aplicaciones Web

1. Principios de arquitectura de software

La arquitectura de software define la organización fundamental de un sistema: sus componentes, las relaciones entre ellos, el entorno en el que operan y los principios que guían su diseño y evolución. Una buena arquitectura busca equilibrar los requisitos funcionales con los atributos de calidad (rendimiento, seguridad, mantenibilidad, escalabilidad).

Principios fundamentales

- Separación de responsabilidades (Separation of Concerns): cada componente debe encargarse de una única responsabilidad bien definida.
- Bajo acoplamiento y alta cohesión: los módulos deben depender lo mínimo posible entre sí, pero cada uno debe ser internamente coherente.
- Reutilización: diseñar componentes que puedan emplearse en distintos contextos sin duplicar lógica.
- Escalabilidad: capacidad del sistema de crecer (en usuarios, datos o carga) sin rediseñarse por completo.
- Mantenibilidad: facilidad para corregir errores, añadir funcionalidades o adaptar el sistema con el menor esfuerzo posible.
- Principio DRY (Don't Repeat Yourself) y principio KISS (Keep It Simple): evitar duplicación y complejidad innecesaria.

2. Arquitectura por capas

La arquitectura por capas (layered architecture) organiza el sistema en niveles horizontales, donde cada capa ofrece servicios a la capa superior y consume servicios de la capa inferior. Es uno de los patrones más utilizados en aplicaciones web tradicionales.

Capas típicas

- Capa de presentación: interfaz de usuario, responsable de mostrar la información y capturar la interacción del usuario.
- Capa de lógica de negocio (o de aplicación): contiene las reglas y procesos que definen el comportamiento del sistema.

- Capa de acceso a datos: gestiona la comunicación con la base de datos u otras fuentes de persistencia.
- Capa de datos: el propio almacenamiento (base de datos, sistema de archivos, etc.).

Ventajas y desventajas

- Ventaja: facilita el mantenimiento y permite sustituir una capa sin afectar a las demás, siempre que se respeten las interfaces.
- Ventaja: favorece la separación de responsabilidades y el trabajo en equipo por especialidades.
- Desventaja: puede introducir sobrecarga de rendimiento al atravesar varias capas para cada operación.
- Desventaja: un diseño rígido de capas puede dificultar cambios transversales que afectan a varias capas a la vez.

3. Arquitecturas en pizarra, dirigida por eventos y peer-to-peer

Arquitectura en pizarra (Blackboard)

Se basa en un espacio de datos compartido (la "pizarra") al que distintos componentes especializados acceden para leer y escribir información de forma colaborativa, sin comunicarse directamente entre sí. Es útil en problemas complejos sin una solución algorítmica determinista, como sistemas de inteligencia artificial o reconocimiento de patrones.

Arquitectura dirigida por eventos (Event-Driven)

Los componentes se comunican mediante la emisión y consumo de eventos, normalmente a través de un bus de eventos o intermediario de mensajería. Cuando ocurre un evento, los componentes suscritos reaccionan de forma asíncrona.

- Favorece el desacoplamiento entre productores y consumidores de eventos.
- Es especialmente adecuada para sistemas reactivos, de tiempo real o con procesamiento asíncrono (por ejemplo, notificaciones, sistemas de mensajería, IoT).
- Introduce complejidad adicional en la depuración y el seguimiento del flujo de ejecución.

Arquitectura peer-to-peer (P2P)

En este modelo no existe una distinción fija entre cliente y servidor: cada nodo (peer) puede actuar como cliente y como servidor simultáneamente, compartiendo recursos directamente con otros nodos de la red.

- Elimina el punto único de fallo asociado a un servidor central.
- Se emplea en sistemas de compartición de archivos, redes blockchain y aplicaciones de comunicación distribuida.
- Plantea retos de seguridad, control de versiones de datos y coordinación entre nodos.

4. Arquitectura Orientada a Servicios (SOA) y Microservicios

SOA (Service-Oriented Architecture)

SOA organiza el sistema como un conjunto de servicios independientes que se comunican entre sí, normalmente a través de protocolos estándar (SOAP, REST) y con frecuencia coordinados por un bus de servicios empresarial (ESB).

- Los servicios son reutilizables y suelen exponer contratos bien definidos (WSDL, por ejemplo).
- Favorece la integración entre sistemas heterogéneos de una organización.
- Suele implicar servicios de mayor tamaño (granularidad más gruesa) que los microservicios.

Microservicios

Es una evolución de SOA que propone dividir la aplicación en servicios pequeños, independientes y desplegables por separado, cada uno responsable de una funcionalidad de negocio concreta y con su propia base de datos cuando es posible.

- Cada microservicio puede desarrollarse, desplegarse y escalarse de forma independiente.
- La comunicación entre servicios suele hacerse mediante APIs REST, gRPC o mensajería asíncrona.
- Facilita el uso de distintas tecnologías por servicio (políglota), pero incrementa la complejidad operativa (despliegue, monitorización, gestión de fallos distribuidos).
- Requiere prácticas como contenedores (Docker), orquestación (Kubernetes) y patrones de resiliencia (circuit breaker, retry, timeout).

Comparativa SOA vs. Microservicios

Aspecto	SOA	Microservicios
Granularidad	Servicios más grandes	Servicios pequeños y específicos
Comunicación	ESB, SOAP/REST	REST, gRPC, mensajería
Base de datos	Frecuentemente compartida	Independiente por servicio
Despliegue	Conjunto, más acoplado	Independiente por servicio
Complejidad operativa	Moderada	Alta (requiere orquestación)

Tema 4

Diseño de Arquitecturas Web Escalables

1. Escalabilidad en aplicaciones web

La escalabilidad es la capacidad de un sistema para manejar un aumento en la carga de trabajo (usuarios, peticiones, volumen de datos) sin degradar su rendimiento, ya sea aumentando recursos o rediseñando su arquitectura.

Tipos de escalabilidad

- Escalabilidad vertical (scale up): consiste en aumentar la capacidad de un mismo servidor (más CPU, más RAM, mejor disco). Es sencilla de aplicar pero tiene un límite físico y supone un punto único de fallo.

- Escalabilidad horizontal (scale out): consiste en añadir más servidores o nodos que trabajen en paralelo, distribuyendo la carga entre ellos. Es la estrategia preferida en sistemas web modernos por su mayor tolerancia a fallos.

Técnicas y componentes habituales

- Balanceadores de carga (load balancers): distribuyen las peticiones entrantes entre varios servidores.
- Replicación de bases de datos: réplicas de lectura para distribuir consultas y reducir la carga sobre el nodo principal.
- Particionamiento de datos (sharding): dividir los datos en fragmentos distribuidos entre distintos servidores.
- Servicios sin estado (stateless): diseñar los servicios de modo que no dependan de información almacenada localmente en el servidor, facilitando añadir o quitar nodos.
- Colas de mensajes: para desacoplar procesos y absorber picos de carga procesando tareas de forma asíncrona.

2. Patrones de caché en aplicaciones web

El caché almacena temporalmente resultados o datos de acceso frecuente para evitar recalcularlos o volver a consultarlos, reduciendo la latencia y la carga sobre los sistemas de backend.

Niveles de caché

- Caché de navegador (cliente): almacena recursos estáticos (imágenes, CSS, JS) localmente en el dispositivo del usuario.
- Caché en CDN (Content Delivery Network): distribuye copias de contenido estático en servidores geográficamente cercanos al usuario.
- Caché de aplicación: almacena en memoria (por ejemplo, con Redis o Memcached) resultados de operaciones costosas o consultas frecuentes.
- Caché de base de datos: mecanismos internos del motor de base de datos para acelerar consultas repetidas.

Estrategias de caché

- Cache-aside (lazy loading): la aplicación consulta primero el caché; si no encuentra el dato (cache miss), lo obtiene del origen y lo guarda en caché para futuras consultas.
- Write-through: los datos se escriben simultáneamente en el caché y en el almacenamiento persistente, manteniendo ambos sincronizados.
- Write-back (write-behind): los datos se escriben primero en el caché y, posteriormente, de forma asíncrona, en el almacenamiento persistente.
- Políticas de expiración (TTL - Time To Live) y de reemplazo (LRU, LFU) para gestionar qué datos permanecen en caché.

Consideraciones

- El caché mejora el rendimiento, pero introduce el problema de la consistencia de datos (invalidación de caché).
- Debe definirse claramente cuándo y cómo se invalida o actualiza cada dato cacheado.

3. Documentación y evaluación de arquitecturas web

Documentar y evaluar una arquitectura es fundamental para comunicarla a los distintos interesados (stakeholders), justificar decisiones de diseño y verificar que cumple con los requisitos de calidad esperados.

Documentación de arquitecturas

- Vistas arquitectónicas: representaciones del sistema desde distintas perspectivas (lógica, de despliegue, de procesos, de datos), como propone el modelo de vistas 4+1.
- Diagramas UML: diagramas de componentes, de despliegue, de secuencia, entre otros, para representar la estructura y el comportamiento del sistema.
- Registro de decisiones arquitectónicas (ADR - Architecture Decision Records): documentos breves que recogen qué decisión se tomó, por qué y qué alternativas se consideraron.

Evaluación de arquitecturas

- Atributos de calidad a evaluar: rendimiento, disponibilidad, seguridad, mantenibilidad, escalabilidad, usabilidad.
- Métodos de evaluación como ATAM (Architecture Tradeoff Analysis Method), que identifica los puntos de riesgo, sensibilidad y compromiso (trade-off) entre distintos atributos de calidad.
- Pruebas de carga y estrés para validar el comportamiento del sistema bajo condiciones reales o extremas de uso.
- Revisión por pares (peer review) y auditorías arquitectónicas periódicas para asegurar que la arquitectura evoluciona de forma coherente con los requisitos del negocio.

Referencias bibliográficas

1. Microsoft Learn. HttpContext.Session Property (System.Web). <https://learn.microsoft.com/en-us/dotnet/api/system.web.httpcontext.session>
2. Microsoft Learn. ASP.NET Session State Overview. [https://learn.microsoft.com/en-us/previous-versions/aspnet/ms178581\(v=vs.100\)](https://learn.microsoft.com/en-us/previous-versions/aspnet/ms178581(v=vs.100))
3. Microsoft Learn. Session and app state in ASP.NET Core. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state>
4. Richards, M. (2022). Software Architecture Patterns (2nd ed.). O'Reilly Media. <https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/ch03.html>
5. ScienceDirect Topics. Layered Architecture — an overview. <https://www.sciencedirect.com/topics/computer-science/layered-architecture>

6. GeeksforGeeks. Types of Software Architecture Patterns. <https://www.geeksforgeeks.org/software-engineering/types-of-software-architecture-patterns/>
7. Solace. The Ultimate Guide to Event-Driven Architecture Patterns. <https://solace.com/event-driven-architecture-patterns/>
8. SentinelOne. SOA vs Microservices: Understanding the Differences. <https://www.sentinelone.com/blog/microservices-vs-soa-whats-the-difference/>
9. AWS. What's the Difference Between SOA and Microservices? <https://aws.amazon.com/compare/the-difference-between-soa-microservices/>
10. Microsoft Learn / Azure Architecture Center. Cache-Aside Pattern. [https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799(v=pandp.10))
11. Hazelcast. A Hitchhiker's Guide to Caching Patterns. <https://hazelcast.com/blog/a-hitchhikers-guide-to-caching-patterns/>
12. Software Engineering Institute, Carnegie Mellon University. The Architecture Tradeoff Analysis Method. <https://www.sei.cmu.edu/library/the-architecture-tradeoff-analysis-method-2>
13. Wikipedia. Architecture Tradeoff Analysis Method. https://en.wikipedia.org/wiki/Architecture_tradeoff_analysis_method